

Technische Hochschule Mittelhessen
Fachbereich MNI
Praktikum Wirtschaftsinformatik, Sommersemester 2026

Aufgabendatenbank

Projektdokumentation

Betreuung: Prof. Dr. Markus Siepermann; Johannes Kunz
Projektteam: [Name 1] (Projektleitung)
[Name 2]
[Name 3]
Gruppe: [Gruppennummer]
Abgabe: 14. Juli 2026

Inhaltsverzeichnis

1	Kurze Zusammenfassung	2
2	Aufgabenstellung und Zielsetzung	2
3	Ausgangssituation und Recherche	2
3.1	Technologieauswahl	3
4	Anforderungen und Rahmenbedingungen	3
4.1	Funktionale Anforderungen	3
4.2	Nicht-funktionale Anforderungen	4
5	Konzept und Entwurf	4
5.1	Systemarchitektur	4
5.2	Beziehungsmodell	5
5.3	Konsistenzentscheidungen	5
6	Umsetzung	5
6.1	Abläufe aus Sicht der Nutzenden	5
6.2	Umsetzung aller Entitäten	6
6.3	Umsetzung der Join-Tabellen	7
6.4	Suche und Bedienlogik	7
6.5	Qualitätssicherung	7
6.6	Build und Bereitstellung	8
7	Projektorganisation	8
7.1	Arbeitsweise mit GitHub	8
7.2	Kommunikation und Zusammenarbeit	8
7.3	Projektphasen und Meilensteine	9
7.4	Aufgabenverteilung	9
7.5	Abweichungen vom ursprünglichen Plan	9
8	Ergebnis, Grenzen und Weiterentwicklung	9
8.1	Erreichter Stand	9
8.2	Bekannte Grenzen	10
8.3	Priorisierte Verbesserungsmöglichkeiten	10
9	Fazit	10
A	Anhang	11
A.1	Lokale Ausführung	11
A.2	Abschließende Nachweise	11

1 Kurze Zusammenfassung

Ausgangspunkt war ein hochschulweit abgestimmtes relationales Schema zur Beschreibung und Organisation von Übungsaufgaben, für das noch keine nutzbare Anwendung existierte. Das Projekt überführt dieses abstrakte Modell in ein Autorenwerkzeug aus Vue-Frontend, Spring-Boot-Backend und PostgreSQL-Datenbank.

Die Anwendung verwaltet Aufgaben mit typisierten Inhaltsbausteinen, Metadaten, Darstellungstemplates und Validatoren. Aufgaben können als ungeordnete Sammlungen, geordnete Sequenzen oder Varianten derselben Ausgangsaufgabe organisiert werden. Hierarchische Tags und kombinierte Filter unterstützen das Wiederfinden. Das Ergebnis ist kein reines CRUD-System: Beziehungen wie `purpose`, `position` und `root_item_id` wurden in konkrete, konsistente Bedienabläufe übersetzt. Automatisierte Tests, Docker und eine GitHub-Actions-Pipeline ergänzen die Anwendung.

2 Aufgabenstellung und Zielsetzung

Die Aufgabe bestand nicht darin, ein eigenes vereinfachtes Datenmodell zu entwerfen. Das vorgegebene Schema mit Entitäten wie `item`, `item_content`, `item_collection`, `tag`, `validator` und `modifier` sollte fachlich verstanden, technisch abgebildet und für reale Nutzende bedienbar gemacht werden.

Zielgruppe sind in erster Linie Lehrende und Autoren von Übungsaufgaben. Sie sollen Aufgaben erstellen, mit strukturierten Inhalten und Metadaten ausstatten, kombinieren und wiederfinden können. Die Anwendung ist weder ein Lernmanagementsystem noch eine Laufzeitumgebung zur Abgabe und automatischen Korrektur. Validatoren und Modifier werden als Regeln modelliert; ihre Ausführung gehört zu einem späteren externen Dienst.

Die Projektziele waren:

- vollständige Abbildung der zentralen Schemaelemente ohne neue Tabellen,
- verständliche Arbeitsabläufe für Aufgaben, Inhalte und Beziehungen,
- konsistente REST-Schnittstellen zwischen Frontend und Backend,
- persistente Speicherung in PostgreSQL,
- automatisierte Prüfung kritischer Geschäftsregeln,
- technische Vorbereitung auf die übergeordnete Plattform.

3 Ausgangssituation und Recherche

Zu Projektbeginn lagen das Schema, ein Lastenheft und ein Vue-basiertes Frontend-Gerüst der übergeordneten EWiLL-Plattform vor. Viele Beziehungen waren technisch benannt, ihre Bedeutung für die Oberfläche war jedoch nicht vollständig festgelegt. Das Team analysierte deshalb zuerst Primär- und Fremdschlüssel, Kardinalitäten und Beziehungsattribute und klärte offene Punkte in Betreuerterminen.

Besonders relevant waren folgende Ergebnisse:

- Ein `item` trägt Aufgabenmetadaten; der eigentliche Text oder eine Datei liegt in einem oder mehreren `item_content`-Datensätzen.
- `purpose` gehört zur Verbindung `item_contents`, weil es die Rolle eines Inhalts in einer konkreten Aufgabe beschreibt.

- Collections modellieren die vertikale Gruppierung beziehungsweise Sequenz; ihre Reihenfolge liegt in `item_collection_sub_item.position`.
- `root_item_id` ist keine Baum-Elternreferenz, sondern verbindet horizontale Varianten derselben Ausgangsaufgabe.
- Validatoren und Modifier sind wiederverwendbare Beschreibungen. Die Aufgabendatenbank speichert und verknüpft sie, führt sie aber nicht aus.

3.1 Technologieauswahl

Die Auswahl beruhte auf fachlicher Eignung, vorhandener Teamerfahrung und den Integrationsbedingungen der übergeordneten Plattform. Da das bereitgestellte Frontend-Gerüst bereits Vue verwendete und das Team damit vertraut war, wurde es erweitert statt durch ein zweites Framework ersetzt. Auch mit Spring Boot und PostgreSQL bestand bereits Erfahrung. So konnte der Schwerpunkt auf Datenmodell und Bedienlogik liegen.

Tabelle 1: Verwendete Technologien; Versionen gemäß finaler Projektkonfiguration

Bereich	Technologie und Version	Einsatz und Begründung
Frontend	Vue 3.5.34, Vue Router 5.0.6, TypeScript 6.0.3, Axios 1.16.0	Erweiterung des bereitgestellten Vue-Frontends; komponentenbasierte, typsichere Oberfläche und REST-Kommunikation. Das Team hatte bereits Vue-Erfahrung.
UI und Interaktion	Vuetify 4.0.7, Pinia 3.0.4, vuedraggable 4.1.0, CodeMirror 6	Einheitliche Bedienelemente, zentraler Zustand, Drag-and-Drop sowie XML-Bearbeitung der Darstellungstemplates.
Backend	Java 21, Spring Boot 3.2.5, Spring Data JPA	REST-Schnittstellen, Geschäftslogik, Transaktionen und Abbildung des relationalen Schemas. Spring Boot war dem Team bekannt und für die Schichtenarchitektur geeignet.
Datenbank	PostgreSQL 16 Alpine	Native Unterstützung der im Schema verwendeten UUIDs, JSONB-Inhalte, GIN-Indizes und BYTEA-Dateien.
Tests und Build	Vitest 4.1.6, JUnit 5, Mockito, MockMvc, Vite 8.0.12, Maven Wrapper 3.9.6	Automatisierte Prüfung von Frontend, Services und Controllern sowie reproduzierbare Produktionsbuilds.
Betrieb und Plattform	Docker Compose, GitHub Actions, GHCR, Kubernetes/ArgoCD	Reproduzierbarer lokaler Start und automatischer Image-Build. Das Cluster-Deployment wurde vorbereitet, blieb aber von externer Infrastruktur abhängig.

Wichtig ist die genaue Bezeichnung: Eingesetzt wird **Vue 3.5.34**. Die Versionsnummer 5.0.6 gehört zu **Vue Router**, nicht zu Vue selbst.

4 Anforderungen und Rahmenbedingungen

4.1 Funktionale Anforderungen

F1 Aufgaben mit Autor, Lizenz, Typ und optionalem Template verwalten.

- F2 Inhaltsbausteine mit Zweck, Typ, Autor und Lizenz verwalten.
- F3 Text, strukturierte Daten, Bilder und PDF-Dateien speichern und anzeigen.
- F4 Geordnete Sequenzen und ungeordnete Sammlungen bilden.
- F5 Varianten einer Ausgangsaufgabe verwalten.
- F6 Validatoren definieren und Aufgaben zuordnen.
- F7 Hierarchische Tags anlegen, zuordnen und filtern.
- F8 Nach Text, Autor, Typ und Tags suchen.
- F9 Darstellungstemplates bearbeiten und als Vorschau anwenden.

4.2 Nicht-funktionale Anforderungen

- referentielle Integrität und transaktionale Geschäftsoperationen,
- typisierte Verträge zwischen Frontend und Backend,
- verständliche Bedienung trotz abstraktem Datenmodell,
- automatisierte Tests für kritische Erfolgs- und Fehlerfälle,
- reproduzierbarer Start in Containern,
- keine strukturelle Erweiterung des vorgegebenen Schemas.

5 Konzept und Entwurf

5.1 Systemarchitektur

Die Anwendung folgt einer Schichtenarchitektur:

```
Vue-Komponente -> Pinia Store -> ApiAdapter -> Axios -> REST-Controller
                                                    -> Service
                                                    -> Repository
                                                    -> PostgreSQL
```

Controller definieren HTTP-Vertrag und Statuscodes. Services validieren Referenzen, kapseln Geschäftsregeln und Transaktionsgrenzen. Spring-Data- Repositories übernehmen den Datenzugriff. DTOs bilden den API-Vertrag, sodass JPA-Entities mit Lazy Loading und Persistenzdetails nicht direkt serialisiert werden. Im Frontend übersetzt der Store DTOs in UI-Modelle. Die Adapter-Schnittstelle entkoppelt ihn von Axios und erlaubt einen lokalen Dummy-Betrieb.

5.2 Beziehungsmodell

	Vertikal	Horizontal
Bedeutung	Gruppierung verschiedener Aufgaben oder Lernsequenz	Varianten derselben Ausgangsaufgabe
Umsetzung	<code>item_collection</code> , <code>item_collection_sub_item</code> , optional <code>position</code>	<code>item.root_item_id</code> , ergänzend Validatoren und Modifier
Reihenfolge	optional; ausschließlich in der Join-Tabelle	keine

Tabelle 2: Fachliche Trennung der Beziehungsarten

5.3 Konsistenzentscheidungen

Beziehungen mit eigenen Attributen wurden als eigenständige JPA-Entities mit zusammengesetztem Schlüssel modelliert. Das betrifft `item_contents` mit `purpose` und `item_collection_sub_item` mit `position`. Ein einfaches `@ManyToOne` hätte diese fachlichen Attribute verloren.

Das Backend ist die Autorität für Reihenfolgen. Beim Aktivieren einer Sortierung erzeugt es fortlaufende Positionen, beim Deaktivieren setzt es alle Positionen auf NULL, und nach einem Verschieben normalisiert es die Geschwister. Die Oberfläche arbeitet bei mehrstufigen Vorgängen optimistisch; nach einem Teilfehler lädt sie den betroffenen Aggregatbereich erneut. Dadurch bleiben Datenbank und Oberfläche konsistent.

6 Umsetzung

6.1 Abläufe aus Sicht der Nutzenden

Initiales Laden. Beim Öffnen lädt das Frontend Referenzdaten wie Autoren, Lizenzen und Typen sowie die Wurzelstruktur. Für ausgewählte Items werden Inhalte und Beziehungen nachgeladen. Die flachen API-Daten werden im Pinia-Store in die Baum- und Editoransicht übersetzt.

Aufgabe und Inhalt erstellen. Der Dialog prüft die Eingaben und sendet zuerst `POST/api/items`. Die zurückgegebene UUID wird anschließend für `POST/api/contents/by-item/itemId` verwendet. Der Backend-Service prüft Autor, Lizenz und Typ, speichert den Content und erzeugt die Verbindung mit ihrem `purpose`. Bei einem partiellen Fehler synchronisiert das Frontend erneut mit dem Server.

Collection bearbeiten. Ein sichtbarer Collection-Knoten besteht aus einem Träger-Item und einem eigenen Collection-Datensatz. Beim Drag-and-Drop sendet das Frontend Quell- und Zielkontext. Der Service ändert die Mitgliedschaft und berechnet bei geordneten Collections die Positionen neu. Das Variantenfeld `rootItemId` bleibt dabei unverändert. Das Löschen einer Collection entfernt nicht automatisch ihre bisher enthaltenen Aufgaben.

Variante erstellen. Eine neue Variante ist ein eigenständiges Item, dessen `root_item_id` auf die Ausgangsaufgabe verweist. Sie besitzt eigene Inhalte und Metadaten. Das Variantenpanel lädt und zeigt alle Items derselben Aufgabenfamilie unabhängig von ihrer Collection-Zugehörigkeit.

Tags, Validatoren und Templates. Tag- und Validator-Zuordnungen werden über eigene REST-Endpunkte gespeichert. Das Frontend baut aus der flachen Tag-Liste rekursiv einen Baum. Templates werden als XML-Text gespeichert; das Frontend liest die Reihenfolge der `purpose`-Platzhalter und ordnet damit die Inhaltsvorschau.

6.2 Umsetzung aller Entitäten

Tabelle 3 trennt bewusst Datenbank, Backend und Frontend. Der Status teilweise bedeutet, dass das Schemaelement berücksichtigt ist, aber kein vollständiger eigener Bedienablauf vorliegt.

Tabelle 3: Umsetzung der Entitäten auf allen Ebenen

Entität	Datenbank und Bedeutung	Backend	Frontend und Status
author	UUID, Bezeichnung und E-Mail; verantwortlich für Item oder Content, aber kein Benutzerkonto.	JPA-Entity, Repository und Referenz- Controller; Schreiboperationen prüfen die ID.	Auswahl und Neuanlage in den Metadaten. Für die erste Iteration vollständig umgesetzt.
license	Wiederverwendbare Lizenz; Item und Content dürfen eigene Lizenzen besitzen. Fremdschlüssel verhindern das Löschen verwendeter Einträge.	Entity, Repository, Lese- und Erstellendpunkte, Referenzvalidierung.	Auswahl und Neuanlage für Item beziehungsweise Content. Umgesetzt.
item_type	Klassifiziert eine Aufgabe. Verbindung zu erlaubten Content-Typen über <code>item_content_types</code> .	Entity, Repository und Referenzendpunkte; JPA-Beziehung zu Content-Typen.	Auswahl, Neuanlage und Suchfilter umgesetzt; automatische Einschränkung kompatibler Content-Typen noch nicht vollständig in der UI.
item_content_type	Beschreibt Format beziehungsweise fachliche Art eines Inhaltsblocks.	Entity, Repository, Referenzendpunkte und Prüfung im Content-Service.	Auswahl und Neuanlage im Content-Editor. Umgesetzt.
item_representation_template	Optional vom Item referenzierter XML-Template-Text; trennt Inhalte von ihrer Anordnung.	Entity, Repository, Service und CRUD-Controller.	CodeMirror-Editor, sortierbare Purpose-Bausteine, Speichern und Vorschau. Umgesetzt als projektspezifische erste Template-Sprache.
tag	Hierarchisches Schlagwort mit Self-Reference <code>parent_tag_id</code> ; beim Löschen bleibt ein Kind als Wurzel erhalten.	Entity, Repository und Controller; flache DTO-Liste, Anlage und Löschen.	Rekursiver Baum, Pfade, Zuordnung, Entfernung und Filter. Item-Tags umgesetzt; eigener Content-Tag-Workflow offen.
validator	Wiederverwendbare Prüf- oder Restriktionsbeschreibung.	Entity, Repository, Service, CRUD und Endpunkte für die Item-Zuordnung.	Editor sowie Zuordnen und Entfernen an Aufgaben. Speicherung umgesetzt, Ausführung bewusst extern.
modifier	Regel zur Transformation oder Erzeugung einer Variante.	Entity und Repository vorhanden; Item-Mapping und IDs im DTO/Service berücksichtigt.	Kein vollständiger Modifier-Editor und keine Ausführung. Strukturell vorbereitet, teilweise umgesetzt.
item	Zentrale Aufgabenentität mit Autor, Lizenz, Typ, optionalem Template und <code>root_item_id</code> für Varianten.	Entity, Repository, CRUD-Service/-Controller, Suche, Varianten-, Tag- und Validatoroperationen.	Baumknoten, Editor, Metadaten, Inhalte, Tags, Validatoren und Varianten. Kernfunktion umgesetzt.

Entität	Datenbank und Bedeutung	Backend	Frontend und Status
item_content	Eigenständiger Content mit Autor, Lizenz, Typ, JSONB oder BYTEA sowie Zeitstempel.	Entity, Repository, Service und Controller; CRUD, Multipart-Upload und Blob-Download.	Content-Liste und Editor; Text, Bildvorschau und Datei-Download. Kernworkflow umgesetzt; Wiederverwendung desselben Contents in mehreren Items nicht als eigener UI-Workflow.
item_collection	Eigene UUID, Träger-Item und order-Kennzeichen.	Entity, Repository, transaktionaler Service und Controller; Konvertierung ist idempotent.	Darstellung als Ordner mit collectionId, Kindern und Sortierstatus. Umgesetzt.
item_collection_sub_item	Mitgliedschaft zwischen Collection und Item; position ist nur bei Sequenzen gesetzt.	Eigene Entity mit Composite Key; Hinzufügen, Entfernen, Verschieben und Normalisieren.	Drag-and-Drop mit vue-draggable; Reihenfolge kommt abschließend vom Backend. Umgesetzt.

6.3 Umsetzung der Join-Tabellen

Tabelle 4: Bedeutung und Status der Join-Tabellen

Tabelle	Umsetzung
item_contents	Verbindet Item und Content und trägt purpose . Deshalb eigene JPA-Entity mit Composite Key. Backend und Content-Editor verwenden die Beziehung vollständig.
item_collection_sub_item	Verbindet Collection und Item und trägt position . Eigene JPA-Entity; Service und Drag-and-Drop verwenden sie vollständig.
item_tags	Many-to-many-Zuordnung ohne Zusatzattribute. JPA-Set, Zuordnungsendpunkte, Chips, Baumwahl und Filter sind umgesetzt.
item_content_tags	In Entity, DTO und Service berücksichtigt. Ein gleichwertiger eigener Content-Tag-Editor wurde nicht fertiggestellt.
item_validator	Many-to-many-Verbindung; Backend-Endpunkte und Validatorpanel sind für Speicherung und Zuordnung umgesetzt.
item_modifier	Item-Zuordnung im Datenmodell und Backend-Mapping vorbereitet; vollständiges CRUD und UI fehlen.
item_content_types	Kompatibilitätsbeziehung zwischen Aufgaben- und Content-Typ. In SQL und JPA modelliert; automatische UI-Filterung ist offen.

6.4 Suche und Bedienlogik

Der Item-Endpunkt akzeptiert optionale Kriterien. Das Backend kombiniert sie über JPA Specifications und bezieht JSONB-Inhalte in die Textsuche ein. Das Frontend kombiniert Suchtext, Autor, Aufgabentyp und Tag-Auswahl und kann die Filter gemeinsam zurücksetzen. Hierarchische Tags werden von der API flach übertragen und erst im Frontend zum Baum aufgebaut. Dadurch bleibt der REST-Vertrag einfach und die Darstellung flexibel.

6.5 Qualitätssicherung

Zum finalen Stand liefen 107 Frontend- und 92 Backend-Tests erfolgreich. Frontend-Tests prüfen unter anderem Store-Aktionen, DTO-Transformation, Baumvalidierung, Collections, Varianten, Suche und Resynchronisation nach partiellen API-Fehlern. Backend-Tests prüfen Item-, Content-, Collection-, Validator- und Template-Services. MockMvc-Tests kontrollieren zentrale Endpunkte

und HTTP-Statuscodes. Zusätzlich waren TypeScript-Typcheck und Vite-Produktionsbuild erfolgreich.

Der frühere leere Spring-Kontexttest wurde entfernt, weil er ohne fachliche Assertion lediglich eine lokal laufende PostgreSQL-Instanz verlangte. Die fachlichen Service- und Controller-Tests bleiben datenbankunabhängig. Ein vollständiger Integrationstest gegen reales PostgreSQL, beispielsweise mit Testcontainers, ist eine sinnvolle nächste Ausbaustufe.

6.6 Build und Bereitstellung

Docker Compose startet PostgreSQL, Backend und Frontend gemeinsam. GitHub Actions baut nach Push auf `main` je ein Backend- und Frontend-Image und veröffentlicht beide mit `latest`- und Commit-SHA-Tag in GHCR. Das SHA-Tag ermöglicht einen reproduzierbaren Deploymentstand.

Eine Kubernetes-`values.yaml` wurde als Integrationsvorbereitung erstellt. Das geplante produktive Deployment über ArgoCD konnte nicht abgeschlossen werden: Die dafür zuständige Plattformgruppe führte ihr Projekt nicht weiter und konnte keine nutzbare Cluster-/ArgoCD-Integration bereitstellen. Dies ist eine externe Abhängigkeit, keine lokal lösbare Anwendungsfunktion.

7 Projektorganisation

7.1 Arbeitsweise mit GitHub

GitHub war nicht nur Quellcodeablage, sondern das zentrale Koordinationswerkzeug. Anforderungen und Fehler wurden als Arbeitspakete in einem GitHub Project erfasst und abhängig vom Arbeitsstand zwischen *In progress*, *In review* und *Closed/Done* verschoben. Dadurch waren Zuständigkeit und Fortschritt für das gesamte Team sichtbar.

Die Umsetzung erfolgte auf Feature- und Fix-Branches. Pull-Requests machten Änderungen nachvollziehbar und ermöglichten Review vor dem Merge. Für übergreifende Änderungen an Store, API-Adapter und DTOs wurden Tests und Typprüfung vor der Integration ausgeführt. Merge-Konflikte traten insbesondere bei parallel bearbeiteten zentralen Dateien auf; kleinere Arbeitspakete und häufigere Integration reduzierten dieses Risiko.

7.2 Kommunikation und Zusammenarbeit

Das Team traf sich abhängig von Fortschritt und Klärungsbedarf ein- bis zweimal pro Woche. In diesen Terminen wurden offene fachliche Fragen, Schnittstellen zwischen Frontend und Backend, Blockaden und die nächsten Arbeitspakete abgestimmt. Zusätzliche kurze Absprachen erfolgten bei Integrationsproblemen.

Einige komplexe Stellen wurden in gemeinsamen Pair-Programming-Sitzungen bearbeitet. Dazu gehörten insbesondere schichtübergreifende Abläufe, bei denen Frontend-Modell, REST-DTO und Backend-Regel gleichzeitig verstanden werden mussten. Pair Programming diente hier sowohl der schnelleren Fehleranalyse als auch dem Wissenstransfer innerhalb des Teams.

7.3 Projektphasen und Meilensteine

Phase	Zeitraum	Ergebnis
Analyse Grundaufbau	Ende April–Mai Mai	Schemaanalyse, Abgrenzung, Technologieentscheidung Datenbank, Backend- und Frontend-Grundstruktur, Docker
Kernfunktionen	Juni	Items, Contents, Collections und REST-Integration
Erweiterungen	Juni–Juli	Varianten, Validatoren, Templates, Tags und Suche
Stabilisierung	Juli	Tests, Fehlerbehandlung, CI/CD und Dokumentation
Abgabe	14. Juli	Bereinigter Stand, Dokumentation und Präsentation

Tabelle 5: Projektphasen

7.4 Aufgabenverteilung

Person	Nachweisbarer Schwerpunkt
[Name 1]	[Projektleitung, Koordination und konkrete Beiträge]
[Name 2]	[Konkrete Frontend-, Backend- oder Datenbankbeiträge]
[Name 3]	[Konkrete Frontend-, Test- oder Deploymentbeiträge]

Tabelle 6: Aufgabenverteilung; vor Abgabe anhand des GitHub Projects und der Pull-Requests zu konkretisieren

7.5 Abweichungen vom ursprünglichen Plan

Das Team wurde im Projektverlauf von fünf auf drei aktive Personen reduziert. Der Umfang musste deshalb priorisiert und Aufgaben mussten schichtübergreifend neu verteilt werden.

Mehrere im Lastenheft vorgesehene Punkte wurden bewusst nicht oder nur teilweise umgesetzt:

- **KI-Unterstützung:** Die im Lastenheft erwogene KI-Funktion wurde zugunsten eines konsistenten Kernsystems aus Datenmodell, Editor und Organisation zurückgestellt. Es fehlte außerdem eine abschließend definierte fachliche Aufgabe, anhand derer Nutzen und Qualität hätten geprüft werden können.
- **Modifier:** Datenbank, Entity, Repository und Item-Zuordnung sind strukturell vorbereitet. Ein vollständiger Editor sowie eine Ausführungslogik wurden nicht umgesetzt, weil die Laufzeittransformation zu einem externen Dienst gehört und Kernfunktionen priorisiert wurden.
- **Authentifizierung:** Das Ausgangsfrontend enthält Ansätze für Benutzer- und Authentifizierungsfunktionen. Für die produktive Integration fehlten jedoch die verbindlichen Schnittstellen und Identitätsdaten der übergeordneten Plattform. Die erste Iteration arbeitet daher ohne rollenbasierte Zugriffskontrolle.
- **Kubernetes/ArgoCD:** Container-Images und Values-Konfiguration sind vorbereitet. Ein tatsächliches Deployment war nicht möglich, nachdem die zuständige Plattformgruppe ihr Projekt nicht fortführte und keine Zielinfrastruktur zur Integration bereitstellen konnte.

8 Ergebnis, Grenzen und Weiterentwicklung

8.1 Erreichter Stand

Vollständig demonstrierbar sind Aufgaben- und Contentverwaltung, Text- und Dateiinhalte, Referenzdaten, Darstellungstemplates, geordnete und ungeordnete Collections, Drag-and-Drop, Varianten, Item-Tags, Validator-Zuordnung sowie kombinierte Suche und Filter. Datenbank, REST-

Backend und Frontend sind über diese Kernabläufe integriert; der Stand ist automatisiert getestet und lokal über Docker startbar.

8.2 Bekannte Grenzen

- Validatoren und Modifier werden gespeichert, aber nicht ausgeführt.
- Für Modifier existiert kein vollständiger Verwaltungseditor.
- Content-Tags sind backendseitig berücksichtigt, aber nicht als gleichwertiger eigener UI-Workflow ausgebaut.
- Die Kompatibilitätsbeziehung `item_content_types` schränkt die Content-Auswahl noch nicht automatisch ein.
- Das Schema erlaubt die Wiederverwendung eines Contents in mehreren Items; die Oberfläche bietet dafür noch keinen eigenen Auswahlablauf.
- Die Baumansicht behandelt die Struktur in der ersten Iteration als eindeutige Position, obwohl das relationale Schema Mehrfachzuordnungen zulässt.
- Zentrale Authentifizierung, produktives Deployment und externe Regelausführung fehlen wegen der beschriebenen Plattformabhängigkeiten.
- Ein Ende-zu-Ende-Test gegen eine echte PostgreSQL-Instanz ist noch offen.

8.3 Priorisierte Verbesserungsmöglichkeiten

1. Integrationstests mit PostgreSQL über Testcontainers,
2. zentrale Authentifizierung und Rollen für Autoren,
3. vollständiger Modifier- und Content-Tag-Workflow,
4. automatische Kompatibilitätsfilter für Content-Typen,
5. explizite UI für die Wiederverwendung von Contents und Mehrfachzuordnungen,
6. Anbindung eines externen Validator-/Modifier-Dienstes,
7. danach eine klar definierte und evaluierbare KI-Unterstützung,
8. produktives ArgoCD-Deployment, sobald Zielplattform und Zugang bestehen.

9 Fazit

Das Projekt hat das vorgegebene Schema nicht vereinfacht oder umgangen, sondern seine zentralen fachlichen Beziehungen in ein nutzbares Autorenwerkzeug übersetzt. Besonders die Trennung von Item und Content, die Unterscheidung von Collections und Varianten sowie die konsistente Positionsverwaltung gehen über einfache CRUD-Masken hinaus.

Nicht erreichte Punkte werden transparent von den implementierten Funktionen getrennt. Der bestehende Stand bildet eine belastbare erste Iteration: Kernabläufe sind über Datenbank, Backend und Frontend integriert, getestet und containerisiert. Die offenen Erweiterungen können auf den vorhandenen Entitäten und Schnittstellen aufbauen, sobald die fachlichen beziehungsweise plattformseitigen Voraussetzungen vorliegen.

Literatur

- [1] Vue.js Documentation. <https://vuejs.org/>
- [2] Vuetify Documentation. <https://vuetifyjs.com/>
- [3] Pinia Documentation. <https://pinia.vuejs.org/>
- [4] Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- [5] PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- [6] Docker Documentation. <https://docs.docker.com/>
- [7] Kubernetes Documentation. <https://kubernetes.io/docs/>
- [8] THM E-Learning-Plattform: bereitgestelltes Frontend, Datenschema, Lastenheft und interne Abstimmungen.

A Anhang

A.1 Lokale Ausführung

```
docker compose up --build
# Frontend: http://localhost:8085
# Backend:  http://localhost:8080
# Dummy:    cd frontend && npm run dev:dummy
```

A.2 Abschließende Nachweise

Vor der finalen PDF-Erstellung sind zu ergänzen:

- Namen, Gruppennummer und konkrete Aufgabenverteilung,
- die fünf Anwendungsscreenshots unter `docs/screenshots/01` bis `05`,
- Screenshot des GitHub Projects als `06-github-project.png`,
- optional ein aussagekräftiger Pull-Request als `07-pull-request.png`,
- ein lesbares ER-Diagramm und der endgültige Repository-Link.